MARLOW — Design Doc

Oct 27, 2026 · @SIVA SURYA CHANDRAN

Overview

MARLOW is a screenless, binaural heist radio-drama you play by voice: you are the handler in the van, Marlow is the operative inside, and she is the only one who can see.

Tagline: You're the voice in her earpiece. She's the only one who can see.

Pitch: Muse understands what you say. A deterministic game engine decides what actually happens. Your only map is what your partner can tell you.

Core loop (one turn):

1. Listen: the building plays around you in 3D audio, heard through Marlow's collar mic.
2. Ask or order: push-to-talk and speak naturally ("peek the left door, quietly").
3. Muse turns your words into one legal action, or asks a clarifying question.
4. The engine resolves the action, moves the guard, and decides what Marlow perceives.
5. You hear the result: Marlow's line on the earpiece plus positioned sounds in the world.

Three things the demo must prove:

- An ambiguous order gets a clarifying question, never a guess.
- An invalid order leaves the world unchanged.
- The same logged actions replay to identical outcomes.

Event: Meta Global AI Developer Hackathon, Indie track. Submission due Nov 4, 2026, 9:00 PM ET.

Why it exists

Most players play alone, and AI game companions are crowded, so MARLOW competes on format: a screenless heist where the AI's limits are enforced by code.

Numbers worth using (secondary reports; cite as "reported by" with the survey population):

Claim	Figure	Source
Players who prefer single-player, 22 countries, Q2 2025	56% (US: 65%)	games.gg summary of 34,000-player study
US PC/console gamers spending 75–100% of play time alone	Over half	Statista / Vorhaus 2024
Players with disabilities worldwide	About 400 million (Microsoft estimate)	VoxBooster roundup
Disabled gamers who skipped a game lacking accessibility	65%	AbleGamers 2024 via accessibility business case
Disabled gamers who abandoned favourite games	39%	ShaneTheGamer roundup (Samsung UK)

Do not use the 250M "visually impaired gamers" figure or WHO's 2.2B vision-impairment figure as a gamer count.

Competitors and how MARLOW differs:

Product	What it does	MARLOW's difference
Razer Project AVA	24/7 desk companion that sees your screen, coaches and cheers	MARLOW is a game, not a companion bolted onto other games
Radio General / There Came an Echo	Voice-commanded strategy with a visible map	No screen and no map by design
A Blind Legend	Audio-only action game; you are the blind hero	You are the handler; your partner is the one who perceives
Whispers from the Star	Open-ended real-time AI conversation with a stranded astronaut	Turn-based, deterministic engine, Marlow's knowledge kept separate from the true map
GamerAstra	Research framework making existing games accessible to blind players	Shipped game designed screenless from the start

Don't claim "first voice + AI partner ever". The wedge is the deterministic engine plus verification.

Meta hackathon fit

The Indie track is judged on creativity, technical ambition, and whether it works; no separate theme score is listed, so the working demo decides, though theme still shapes how judges read the project.

Theme framing (honest version): Build for Your World, through the builder's real domain. "I'm a validation engineer: I build systems that must obey a spec. In MARLOW, Muse interprets the player, and a deterministic engine checks every action and owns every consequence."

Why it fits Meta's direction: Meta calls Muse the core of its personal-superintelligence push, with work on preference memory and long-term continuity (36Kr, Connect 2026). Zuckerberg named games among Muse Spark's strengths (Yahoo Finance). Meta also markets blind and low-vision accessibility on its glasses (Be My Eyes, Mar 2026; UploadVR).

Tool map:

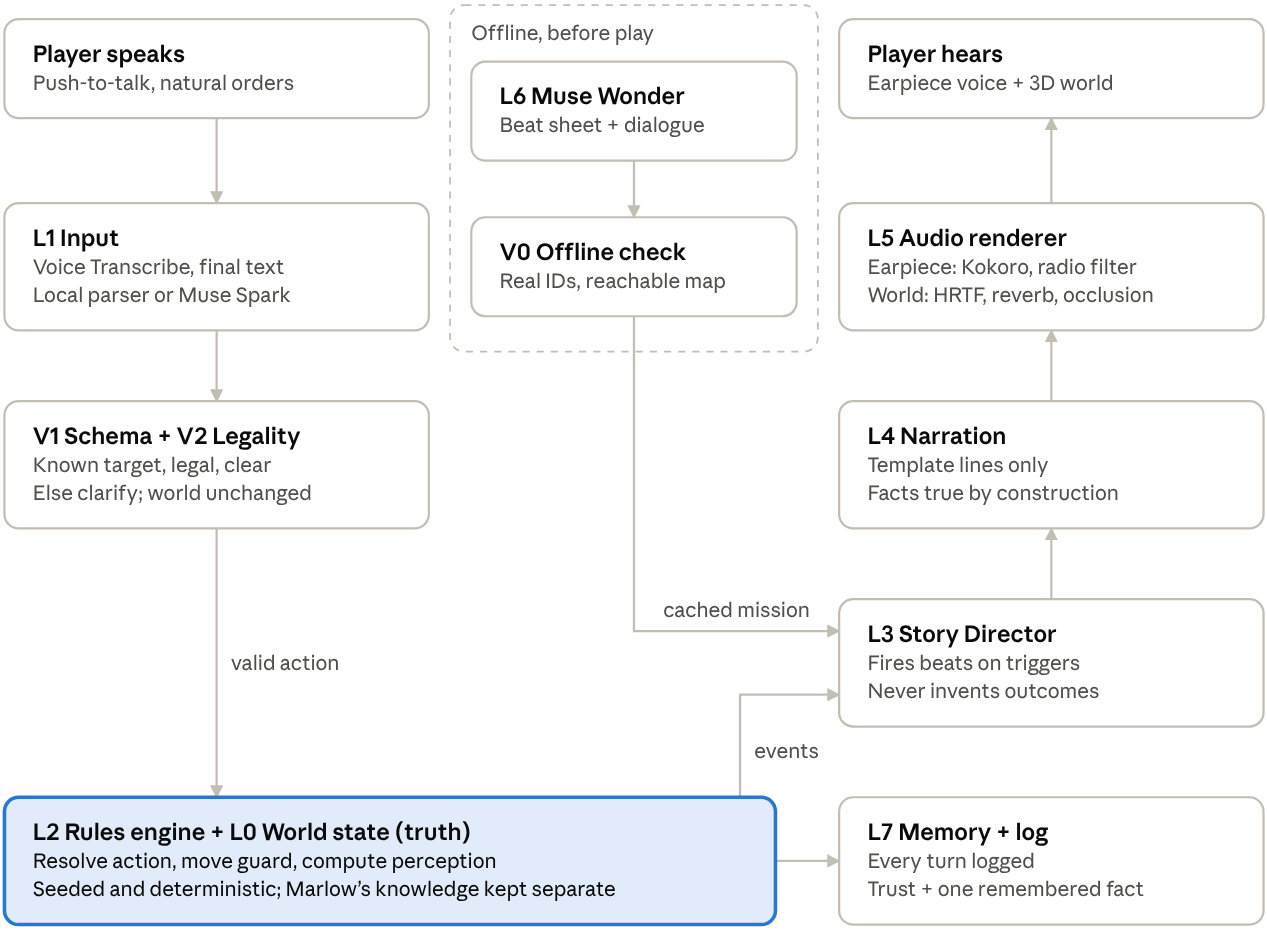
Tool	Job in MARLOW	Status
Muse Voice Transcribe	Push-to-talk speech to text; partial results for on-screen display only	Core (confirm access in Discord)
Muse Spark	Turns natural or ambiguous orders into one action JSON, or a clarify request with candidate IDs (low reasoning-effort setting)	Core
Muse Wonder	Selects and orders beats and cue IDs offline; any drafted lines are hand-reviewed; validated and cached	Core (hand-written backup exists)
Muse Image	Renders room art from engine state for the post-run case file	Stretch
SAM 3.1 + Spark vision	Verifies generated art matches engine state (props present, right side); regenerate on mismatch	Stretch
Muse Code	Development tool	Dev only
Meta open weights	No forced use	Not used

Not used, on purpose: DeepSeek or other non-Meta models for story (muddies the Meta story; rules unverified), MMS-TTS and MusicGen (non-commercial licenses).

System architecture

Input runs down the left column into the engine; results run up the right column to the player's ears; the offline author feeds the Director a validated, cached mission.

Only the engine changes the world; every model output passes a check first



MARLOW architecture · 8 layers, 2 runtime checks + 1 offline check

The engine box is the only place the world changes. Everything a model produces is either checked before it reaches the engine (V1, V2) or checked before play (V0).

Layers in detail

Eight layers, one rule: only the engine (L0 + L2) changes the world; every other layer reads it, interprets input for it, or renders it.

L0 — World state (the only truth)

- One JSON object: rooms and links, doors (open, closed, locked), props with tags (hide, noisy, lootable), guard with a fixed patrol route, objective.

- Turn counter, alert level 0–3, trust 0–100 (rules in Story system). Spatial model: each room has a grid position, each door and prop has a side within its room (left, right, ahead, behind), and Marlow has a facing; direction and distance (near, mid, far by room hops) are computed from these.
- **Marlow's knowledge** is a separate structure holding only what she has perceived. The player learns only through it.
- A seeded random generator picks the guard's starting step, template variants and sound variations, so any run replays exactly; the replay test also catches hidden nondeterminism (timers, async ordering).

L1 — Input understanding

- Push-to-talk audio → Muse Voice Transcribe. Partial transcripts may be shown on screen; actions use the final transcript only.
- **Local parser** handles unambiguous commands with plain code: wait, stop (= wait), go back (= move to the previous room), yes, no, repeat. No model call.
- **Muse Spark (documented low reasoning-effort setting)** handles natural or ambiguous orders and returns exactly one action, or a clarifying question. No compound commands in V1. Every request carries a turn ID; a reply that arrives after its turn timed out is discarded.

L2 — Rules engine (deterministic)

- Checks preconditions, resolves the action, then advances the guard one step on its route.
- **Perception model:** sight follows room and door state; sound travels the room graph, quieter with distance and closed doors.
- Output: an event list. Perception events update Marlow's knowledge and drive audio, spoken direction and the text log from the same source. Every cue the player hears is a perception event.

L3 — Story Director

- Holds the beat sheet: beats with trigger conditions on engine state (first time alert reaches 2, ledger taken, turn 8 without the objective, trust below 30).
- Fires at most 1–2 beats per turn. Tension-curve pacing is behind the cut line.
- May add dialogue, music changes, ambience, and pre-declared world events from the level file. It can never invent a guard, a door or an outcome.

L4 — Narration

- Every factual line comes from templates: 3–5 variants per event type, true by construction.
- No live free-form flavor lines in V1. Clarifications are templates too: Spark names the candidate IDs, the template speaks them. Every template line is pre-rendered to audio offline with Kokoro, so there is no TTS at runtime.

L5 — Audio renderer

- Earpiece bus: Marlow's Kokoro voice, mono, centred, radio-filtered.
- World bus: binaural sounds positioned from perception events only, with room reverb, occlusion and distance.
- Music stems by alert level; ducking under Marlow's voice; a cue scheduler so danger cues never collide.

L6 — Mission Author (offline)

- Muse Wonder selects and orders beats and cue IDs for a hand-made building, as JSON tied to entity IDs. Any dialogue it drafts is edited by hand before caching: reviewed, not automatically verified.
- Validated by V0, cached, shipped as a file. A hand-written backup mission always exists.

L7 — Memory and log

- Every turn is logged: transcript, parsed action, events, lines played.
- Trust and one remembered fact ("rushed the stairs last run") live in app storage, not model memory.
- Debrief facts come from the log via templates.

Verification layers

No model output reaches the player or the world without passing a code check; V1 keeps four checks, numbered V0–V3.

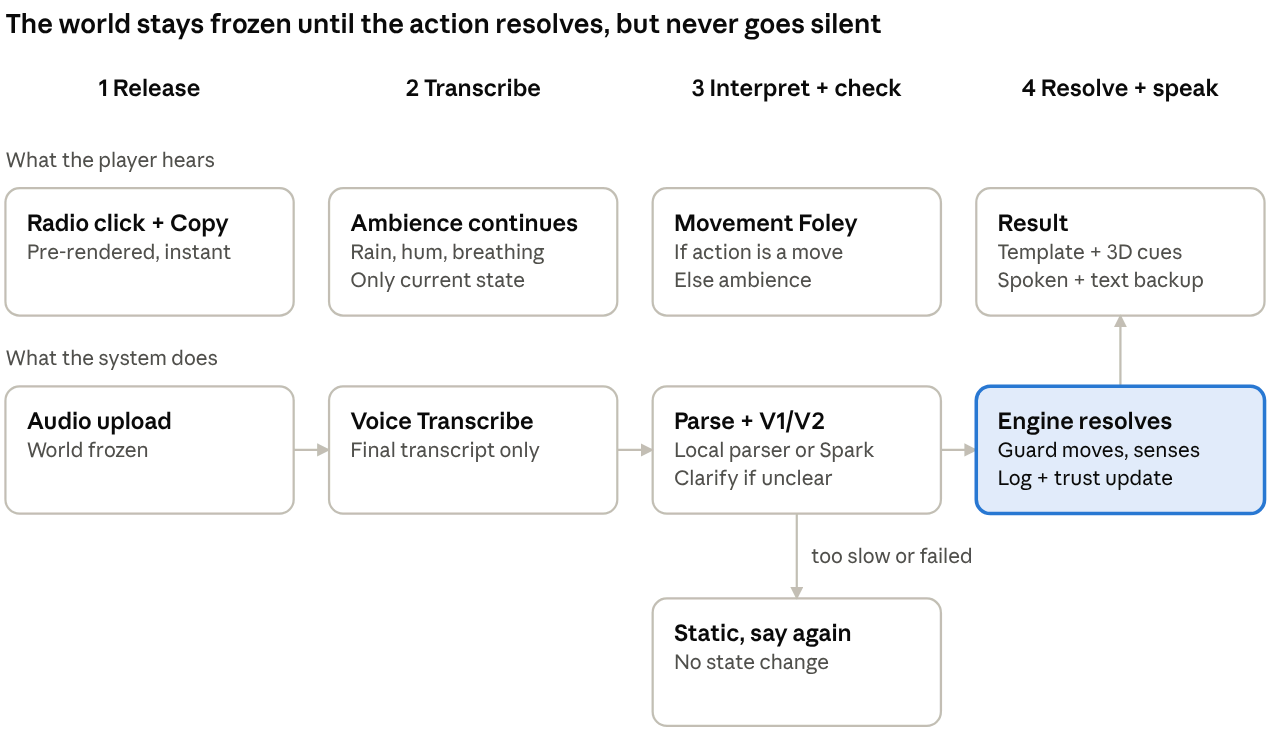
Check	Runs at	What it verifies	On failure
V0 Offline mission check	Load / build time	Every beat trigger and cue references real entity IDs; all rooms reachable (BFS); ledger takeable and exit reachable; scheduled events point to real things. Does not prove dialogue true; lines are hand-reviewed	Regenerate once, else ship the hand-written mission
V1 Schema	Every Spark reply	Output matches the action JSON schema exactly	Retry once, then Marlow: "Say again, boss?"
V2 Legality + knowledge	Before the engine	Target exists in Marlow's knowledge (not just the world); action allowed now; reference unambiguous	In-character clarify or refusal: "Which door? I see two." World unchanged
V3 Regression harness	Development	50 recorded utterances scored separately for transcription and intent; seeded replays → identical event lists; invalid actions accepted (must be 0)	Fix before demo; report as metrics
Image verify (stretch)	Case-file generation	SAM masks + Spark confirm generated art shows the props the engine placed, on the right side	Regenerate; after two fails use fallback art

Removed on purpose: checks on free-form flavor lines and debriefs. A validator can confirm a sentence mentions real IDs, not that the relationships it states are true, so factual speech is template-only.

Truth direction: engine → narration, engine → audio, engine → image. Nothing flows back from a model into world state except a validated action. Every async reply carries a turn ID, and replies for a timed-out turn are discarded. The earlier V3/V4 checks on free-form lines were removed, so numbering now runs V0–V3.

Turn lifecycle and latency

A turn freezes the world from the moment the player speaks until the engine resolves the action; the player always hears something while models work.



One turn · 4 phases, 1 timeout branch

Only the engine box changes state. Filler sound reflects the current state only, so it never leaks an outcome before the engine decides it.

Latency rules:

- Simple commands skip Spark via the local parser.
- Spark runs in Instant mode.
- Partial transcripts are display-only; actions use the final transcript.
- Hard timeout reads in-fiction as a bad signal; nothing changes.

- Targets are guesses until measured on day 2: aim for the result in about 3 seconds after release.

Audio design

Sound is the interface: the player hears the building through Marlow's collar mic in binaural 3D, and every sound comes from a perception event.

Two buses:

Bus	Sound	Purpose
Earpiece	Marlow's voice: mono, centred, close, band-pass radio filter (~300 Hz–3.4 kHz) + faint static	Story and instructions, never confused with the world
World	Binaural via Web Audio HRTF panning, per-room reverb, occlusion, distance	Where things are and how the space feels

Spatial cues that carry meaning:

- Moving sources (a guard's footsteps sweeping in an arc) are easier to locate than static ones; generic HRTFs confuse front and back.
- Room reverb identifies rooms: dry office, long marble echo, metallic vault.
- Occlusion: low-pass filter behind closed doors, clear when opened.
- Distance: quieter and duller, not just quieter.
- Footstep materials: carpet, marble, metal grating.

Mixing rules:

- Music and world duck about 8–10 dB under Marlow's voice (tune by ear).
- Music stems layer in by alert level 0–3.
- Danger cues outrank everything except Marlow's voice; two important cues never overlap.
- Separate sliders for voice, music and SFX; a mono option.

Accessibility backups: every cue is also spoken ("footsteps, behind left") and written to the text log. Keyboard and text input work alongside voice. 3D audio is immersion, not the only channel.

Open-source stack:

Need	Choice	License
Spatial audio, reverb, filters	Web Audio API (PannerNode HRTF)	Browser standard
Voice (TTS)	Kokoro-82M, run offline to pre-render every template line; the files play through Web Audio for the radio filter. Fallback: a recorded voice bank	Apache 2.0 weights; check runtime deps
Reverb impulse responses	OpenAIR library	Per file, Creative Commons
Sound effects	Freesound (CC0 filter), Sonniss GDC bundles	CC0 / royalty-free
Music	CC0 or royalty-free stems (e.g. Pixabay Music; Incompetech needs credit)	Log every file in CREDITS.md

Avoid: browser system speech (can't be filtered), Resonance Audio (archived repo), any NC-licensed model or asset.

Story system

The story is a radio drama triggered by engine events: Wonder writes it ahead of time, the Director fires it, and it never decides what happened.

Cast (small, so every voice is distinct):

Character	Heard as	Role
Marlow	Earpiece, close and filtered	The operative; your only eyes; trust changes her tone
You	Silent player character	The handler in the van
The guard	World bus, through walls	Threat you track by sound
The client	Phone call in the briefing	Sets the job and the twist

Beat sheet: each beat has an ID, a trigger condition on engine state, template line IDs and optional sound or music cue IDs. Example triggers: mission start, first time alert reaches 2, ledger taken, turn 8 without the objective, trust below 30, extraction.

Director rules:

- At most 1–2 beats per turn.
- Behind the cut line: tension pacing (after quiet turns, a scripted complication; after chaos, a breather).
- State changes only through pre-declared events in the level file (the office phone rings at turn 6).

Let sound carry the plot: a ticking timer, rain getting heavier, a phone buzzing in another room. In audio drama the strongest beats are sounds, not lines.

Memory between runs: trust (one number) and one remembered event. Run 2 opens with a template line built from it: "Last time you rushed the stairs. Patience, okay?"

Trust rules (start at 50, clamped 0–100; the remembered fact is the last event that lowered trust):

Engine event	Trust change
Marlow spotted by the guard	−10
Player orders a move right after Marlow warns	−5
Player waits after Marlow warns	+5
Ledger taken	+5
Exit with alert at 1 or lower	+10

Trust below 30 changes Marlow's template set to a curter tone; it never changes game rules.

Data schemas

Four JSON shapes carry everything; these are starting sketches to refine on day 1.

World state (L0):

```
{
  "seed": 1234,
  "turn": 3,
  "alert": 1,
  "trust": 55,
  "rooms": [{"id": "lobby", "pos": [0, 0], "reverb": "marble", "links":
["door_1"]}],
  "doors": [{"id": "door_1", "between": ["lobby", "hall"], "side": {"lobby":
"ahead", "hall": "behind"}, "state": "closed"}],
  "props": [{"id": "desk_1", "room": "office", "side": "left", "tags":
["hide"]}],
  "guard": {"id": "g1", "route": ["hall", "office", "hall", "stairs"],
"step": 2},
  "marlow": {"room": "lobby", "facing": "north", "hidden": false, "carrying":
null, "prevRoom": null},
  "knowledge": {"rooms": ["lobby"], "doors": ["door_1"], "guardLastHeard":
null},
  "objective": {"item": "ledger", "room": "office", "taken": false,
"exitRoom": "lobby"}
}
```

Action (Spark output, checked by V1/V2):

```
{"action": "peek", "target": "door_1", "stance": "quiet"}
```

or

```
{"clarify": {"candidates": ["door_1", "door_2"]}}
```

Actions in V1: move, peek, listen, wait, hide, take, exit. One per turn. A clarify reply names candidates only; the spoken question comes from a template ("Which door? The one ahead, or on the left?"). No confidence score unless it is calibrated.

Event (engine output):

```
[
  {"type": "door_peeked", "id": "door_1", "sees": ["hall"]},
  {"type": "guard_heard", "id": "g1", "dir": "left", "dist": "near"}
]
```

Beat (Wonder output, checked by V0):

```
{
  "id": "alert_2_first",
  "trigger": {"alert_gte": 2, "once": true},
  "lineIds": ["marlow.alert2.a", "marlow.alert2.b"],
  "cueIds": ["music.tension.on", "sfx.radio.static"]
}
```

Line and cue IDs point to the hand-reviewed template bank and asset list; Wonder never supplies the words at runtime.

Build plan and cut line

Eight days, no slack: the game is L0–L2, the audio and the turn handling, and everything else sits behind a hard cut line.

V1 scope: one hand-made level (about 5 rooms), one guard on a fixed route, one objective (take the ledger, then exit), seven actions, one action per turn, two runs with trust carrying over.

Stack: TypeScript in the browser, Web Audio, Kokoro run offline to pre-render template lines, a thin server holding the Meta API key.

- [ ] Day 1 AM — Go/no-go on Meta API access (Spark, Wonder, Voice Transcribe), logged as a decision; post a request for one blind or low-vision tester
- [ ] Day 1 PM — Typed engine with all seven actions; seeded determinism test
- [ ] Day 2 — One complete voice turn; measure p50 and p95 latency; Kokoro offline pre-render spike (fallback: recorded voice bank)
- [ ] Day 3 — Spatial model + world audio driven by perception events: HRTF, reverb, occlusion, distance
- [ ] Day 4 — Pre-rendered template lines through the radio filter; ack, ambience, timeout, late-reply discard
- [ ] Day 5 — Spark clarify flow, V1/V2 checks, V3 regression harness with split metrics
- [ ] Day 6 — Beat sheet (hand-written; Wonder version only if it passes V0 and hand review) + trust rules
- [ ] Day 7 — Playtests: blindfold smoke test plus the recruited tester in a remote session; collect metrics
- [ ] Day 8 — Demo video + README + CREDITS.md

Hard cut line (only after day 6, if ahead): case file with Image + SAM verification, Wonder debrief, tension pacing, more than one alert music layer, real-city flavor via OpenStreetMap (needs attribution).

Never cut: the validator checks, audio ducking, spoken + text backups for cues, the regression harness.

Cut entirely: compound commands, live flavor lines, procedural missions at runtime, interview-built levels, real-street getaway, Google Maps or real building imagery.

Metrics and demo

Show judges measured numbers from the regression harness and playtests, not adjectives; the targets below are working goals, not results.

Metric	How measured	Working target
Transcription errors	50 recorded commands: transcript vs what was said	Report rate
Intent errors	Same 50, given a correct transcript: parsed action vs expected	⩾ 90% correct
Turn latency	Push-to-talk release → first word of result, p50 and p95	Measure on day 2; aim p50 under ~3 s
Invalid actions accepted	Illegal or unknown targets that reached the engine	0
Clarification rate	Turns where Spark returned candidates instead of an action	Report as is
Replay determinism	Seeded replays producing identical event lists	100%
Playtest completion	Testers who take the ledger and exit, plus quotes	Report as is

Demo video (about 90 seconds, five beats):

1. Black screen, rain; the client's call sets the job.
2. Voice orders; Marlow asks one clarifying question; a guard's footsteps sweep left to right and the player waits.

3. An invalid order: "There's no door there, boss." Nothing changes.
4. Ledger taken, alert music rises, clean exit; run 2 opens with the trust callback line.
5. Split screen: hidden engine map, event log and metrics.

Five beats leave room for the audio to land. Check whether the submission allows 120 seconds.

If a blind or low-vision tester plays it, their reaction goes first in the video.

Risks and decisions

The biggest risks are latency, scope and overclaiming accessibility; each has a fallback.

Risks:

Risk	Fallback
Voice round trip too slow	Local parser for simple commands; instant ack + ambience; timeout reads as "bad signal"
Voice Transcribe not in hackathon access	Browser speech recognition or typed input; go/no-go on day 1 morning (weakens the Meta tool story to two core tools)
Spark misreads orders	Clarify instead of guess; V2 blocks illegal or unknown targets
Wonder output breaks V0	Ship the hand-written mission
Generic HRTF front/back confusion	Moving sources, spoken direction, text log
Accessibility overclaim	Say "designed to be playable without vision"; blindfold is a smoke test only; seek a real blind or low-vision tester
Scope slip	Cut line in the build plan
API overspend	$150 spend alert, per-session call cap, cached assets

Decision log (newest first):

Decision	Why
Template lines pre-rendered offline; no runtime TTS	All factual lines are known ahead; removes in-browser Kokoro risk
take and exit added; stop = wait, go back = previous room	The mission must be completable for V0 to pass
Wonder outputs beat and cue IDs; drafted lines hand-reviewed	V0 can check references, not whether a line is true
Discrete spatial model: room positions, sides, Marlow's facing	Audio direction and distance need position data
Replies tagged with turn ID; late replies discarded	A stale reply must never change the next state
Trust kept as a five-rule table	Run 2 callback depends on it; about an hour of work
Kokoro only for voices	Radio filter needs audio routed through Web Audio; one engine is simpler
No live flavor lines	A checker cannot prove a sentence's relationships are true
World frozen during a turn; no outcome prefetch	Turn-based clarity; the engine is instant anyway
Sounds only from perception events	Audio must never reveal what Marlow hasn't perceived
Wonder output cached, not generated at load	No long load; hand-written backup always ready
Truth flows engine → image, never image → engine	Generated pixels are not geometry
No Google Maps or real building photos	Terms risk, safety optics, unreliable interiors
No DeepSeek for story	Keep the Meta story clean; rules unverified
Name: MARLOW	Two clear syllables, survives speech-to-text, noir tone
Screenless heist over a GTA companion	Companion space is crowded (Razer AVA)

Licenses, costs and sources

MARLOW can be built for $0 out of pocket; the only requirement is a card on file to unlock the $150 Meta API credit.

Costs:

Item	Cost	Note
Meta Model API	$150 free credit	Claim by Nov 4, 9:00 PM ET; card required; set the $150 spend alert
Partner credits (Fireworks, Baseten, Modal)	$100 each, select participants	Not needed
Audio engine, TTS, sounds, music	Free	Open source / CC0
Hosting	Free tier	GitHub Pages, Vercel or Netlify
Headphones, mic	Existing gear	Normal stereo earbuds are the right test device

License rules: prefer CC0 and Apache/MIT; no NC (non-commercial) models or assets; every asset logged in CREDITS.md; README states real places, if any, are settings only and all characters and targets are fictional.

Sources:

Most market figures come from secondary roundups. Before the README, open each link, quote the figure as "reported by" that source with its survey population, and drop any link that no longer opens.

- games.gg — single-player preference study
- Statista — time spent gaming alone (Vorhaus 2024)
- VoxBooster — gaming accessibility statistics 2026
- ShaneTheGamer — gaming accessibility statistics
- Accessibility business case (AbleGamers 2024 data)
- Razer — Project AVA
- Notebookcheck — Project AVA
- GamerAstra (arXiv)
- AppleVis — blind gamers on AI audio description

- 36Kr — Meta Connect 2026 and Muse
- Yahoo Finance — Muse Spark launch
- Be My Eyes — Meta partnership expansion
- UploadVR — Detailed Responses for blind and low-vision users
- MDN — PannerNode panningModel
- Kokoro-82M on Hugging Face
- OpenStreetMap copyright and attribution